SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

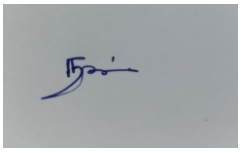
QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I MS Naveen(192371002) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

1. Password Hashing with Salting and Key Stretching
Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

---

2. Implementation of HMAC for Message Integrity
Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

---

3. Simulation of Certificate Chain Validation
Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

---

4. Replay Attack Simulation and Prevention
Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

---

5. Performance Comparison of Digital Signature Algorithms
Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

- **1. Password Hashing with Salting and Key Stretching**

Aim

To design and implement a Python program for securely storing and verifying passwords using **salted hashing and key stretching with PBKDF2**, and analyze how these techniques protect against brute-force and rainbow-table attacks.

Objective

- Generate a unique random salt for every password.
- Hash passwords using PBKDF2 with SHA-256.
- Store only the salt and derived hash instead of the original password.
- Verify passwords securely.
- Understand the importance of salting and key stretching.
- Measure the computational overhead of password hashing.

Theory

Password Hashing

A password should never be stored directly in a database.

Instead:

Password

↓

Hash Function

↓

Password Hash

↓

Database

For example:

Password: MyPassword123

Hash: 8f7a...c92

If an attacker obtains the database, they cannot directly see the original password.

What is Salting?

A **salt** is a random value added to a password before hashing.

Password + Random Salt

↓

PBKDF2

↓

Password Hash

For example:

Password = password123

User 1 salt = A82F91...

User 2 salt = B71C42...

Even if both users have the same password, their resulting hashes will be different.

What is Key Stretching?

Key stretching intentionally makes password hashing computationally expensive.

PBKDF2 performs the hashing operation many times.

For example:

Password

↓

Hash

↓

Hash

↓

Hash

↓

...

↓

Final Derived Key

This makes large-scale password guessing significantly slower.

Python Implementation

Requirements

Python 3 is sufficient because the implementation uses the built-in hashlib and secrets modules.

Create:

password_security.py

Program

```
import hashlib

import secrets

import time

import hmac


# Configuration

ITERATIONS = 600_000

SALT_LENGTH = 16

KEY_LENGTH = 32



def hash_password(password):

    # Generate a cryptographically secure random salt

    salt = secrets.token_bytes(SALT_LENGTH)


    # Derive password hash using PBKDF2-HMAC-SHA256

    password_hash = hashlib.pbkdf2_hmac(

        "sha256",
```

```
        password.encode("utf-8"),
        salt,
        ITERATIONS,
        dklen=KEY_LENGTH
    )

    return salt, password_hash


def verify_password(password, salt, stored_hash):
    # Generate hash using the same salt and parameters
    password_hash = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt,
        ITERATIONS,
        dklen=KEY_LENGTH
    )

    # Constant-time comparison
    return hmac.compare_digest(password_hash, stored_hash)


def main():
    print("PASSWORD HASHING WITH SALTING AND KEY STRETCHING")
    print("-" * 55)
```

```
password = input("Enter password: ")

# Measure hashing time
start = time.perf_counter()

salt, stored_hash = hash_password(password)

end = time.perf_counter()

print("\nPassword successfully processed.")
print("Salt:", salt.hex())
print("Hash:", stored_hash.hex())
print("Hashing time: {:.4f} seconds".format(end - start))

# Password verification
print("\nPASSWORD VERIFICATION")

entered_password = input("Enter password for verification: ")

start = time.perf_counter()

result = verify_password(
    entered_password,
    salt,
    stored_hash
)
```

```
    end = time.perf_counter()

    if result:
        print("Result: Password is CORRECT")
    else:
        print("Result: Password is INCORRECT")

    print("Verification time: {:.4f} seconds".format(end - start))


if __name__ == "__main__":
    main()
```

Sample Output

```
PASSWORD HASHING WITH SALTING AND KEY STRETCHING
--------------------------------------------------
Enter password: MyPassword123

Password successfully processed.
Salt: 4e8c2d9a8f1b73a4c1e5d7a92b3f6c11
Hash: 8b5c2f9e7d3a1b6c4e8f2a9d7c1e5b3f...
Hashing time: 0.1652 seconds

PASSWORD VERIFICATION
Enter password for verification: MyPassword123

Result: Password is CORRECT
```

Verification time: 0.1648 seconds

If the wrong password is entered:

Enter password for verification: wrongpassword

Result: Password is INCORRECT

Analysis

1. Protection Against Rainbow Tables

Without a salt:

password123 → fixed hash

An attacker can create a large precomputed table:

password123 → hash1

qwerty → hash2

admin123 → hash3

This is called a **rainbow-table attack**.

With salting:

password123 + Salt A → Hash A

password123 + Salt B → Hash B

Therefore, the same password produces different hashes.

This makes precomputed rainbow tables much less useful.

2. Protection Against Brute-Force Attacks

An attacker may try:

password1

password2

password3

...

If password hashing takes only a tiny fraction of a second, millions of guesses can potentially be attempted quickly.

PBKDF2 deliberately increases the computation required for each guess.

Therefore:

Normal hash:

1 guess → very fast

PBKDF2:

1 guess → computationally expensive

The attack becomes slower.

3. Performance Overhead

The major disadvantage of key stretching is additional computation.

Example:

Method	Relative Speed	Security for Password Storage
SHA-256 once	Very fast	Not recommended
SHA-256 + salt	Fast	Better but insufficient alone
PBKDF2 + salt	Slower	Suitable when properly configured
bcrypt/Argon2 + salt	Deliberately expensive	Strong password-storage choices

The exact execution time depends on the computer and selected iteration count.

4. Why compare_digest() is Used

The program uses:

hmac.compare_digest()

instead of a normal equality comparison.

This provides a constant-time comparison mechanism, reducing information leakage through timing differences.

Advantages

- Protects passwords if the database is compromised.
- Unique salt prevents identical passwords from having identical hashes.
- PBKDF2 makes password guessing more expensive.
- Uses cryptographically secure random salt generation.
- Password itself is never stored.

Limitations

- Weak passwords can still be guessed.
- PBKDF2 consumes CPU resources.
- Very high iteration counts can affect login performance.
- Password hashing does not protect against phishing or password theft before hashing.
- For new systems, memory-hard password hashing such as Argon2id is often preferred when available.

Conclusion

The experiment demonstrates that **salting and key stretching significantly improve password security**. Salting prevents attackers from efficiently using precomputed rainbow tables, while PBKDF2 increases the computational cost of each password guess. The additional processing time is an intentional security feature.

- **2. Implementation of HMAC for Message Integrity**

Aim

To develop a Python application implementing **HMAC using SHA-256** and analyze how HMAC provides message integrity and authentication compared with simple hashing.

Objective

- Implement HMAC-SHA256.
- Generate a message authentication code using a secret key.
- Verify whether a received message has been modified.
- Compare HMAC with ordinary hashing.

- Understand how HMAC provides authentication and integrity.

Theory

What is Hashing?

A hash function converts data into a fixed-length value.

Message

↓

SHA-256

↓

Hash

Example:

Message:

Hello World

SHA-256:

A591A6D40BF420404A011733CFB7B190...

However, **a normal hash does not prove who generated the message**.

An attacker can modify the message and calculate a new hash.

What is HMAC?

HMAC stands for:

Hash-based Message Authentication Code

It combines:

Secret Key

+

Message

↓

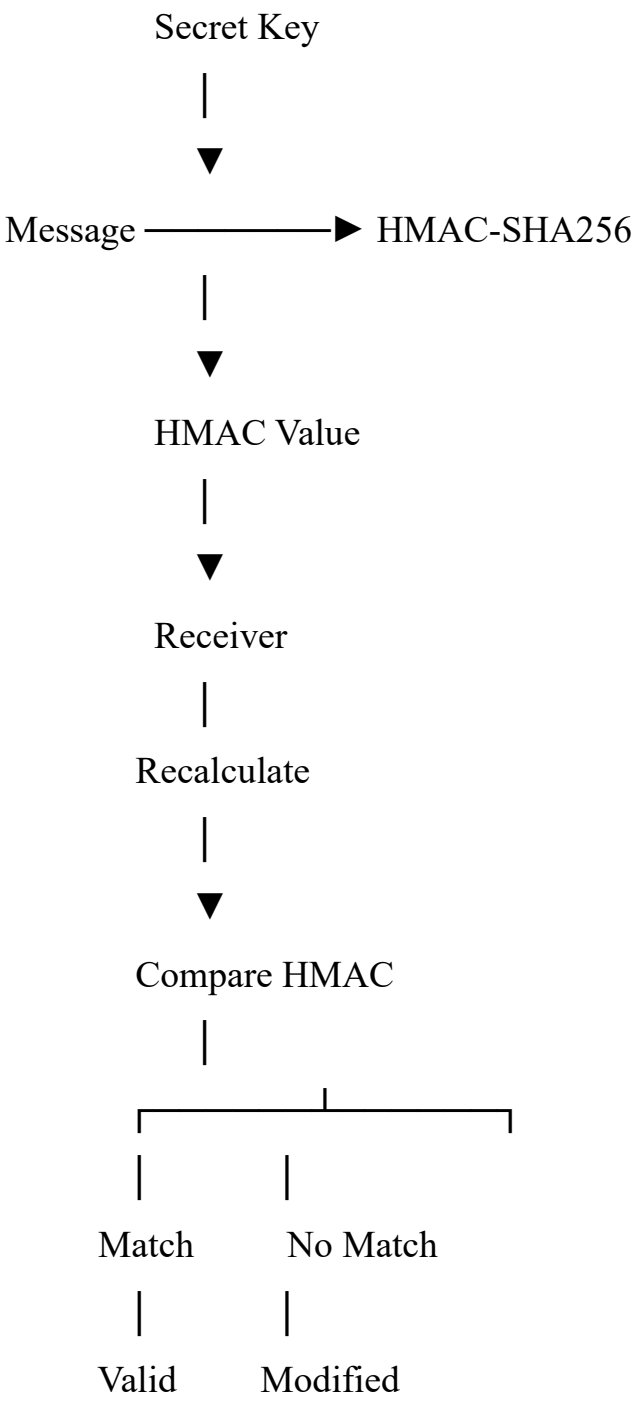
HMAC-SHA256

↓

Authentication Code

Only someone possessing the secret key should be able to generate a valid HMAC.

HMAC Architecture



- **Python Implementation**

Create:

hmac_security.py

Program

```
import hmac
import hashlib
import time


def generate_hmac(message, secret_key):
    return hmac.new(
        secret_key,
        message.encode("utf-8"),
        hashlib.sha256
    ).hexdigest()


def verify_hmac(message, secret_key, received_hmac):
    calculated_hmac = generate_hmac(
        message,
        secret_key
    )

    return hmac.compare_digest(
        calculated_hmac,
        received_hmac
```

```
    )


def main():
    print("HMAC-SHA256 MESSAGE AUTHENTICATION")
    print("-" * 45)

    message = input("Enter message: ")
    key = input("Enter secret key: ")

    secret_key = key.encode("utf-8")

    # Generate HMAC
    start = time.perf_counter()

    generated_hmac = generate_hmac(
        message,
        secret_key
    )

    end = time.perf_counter()

    print("\nGenerated HMAC:")
    print(generated_hmac)

    print(
        "\nGeneration time: {:.6f} seconds".format(
```

```
        end - start
    )
)

# Verification
print("\nMESSAGE VERIFICATION")

received_message = input(
    "Enter received message: "
)

received_hmac = input(
    "Enter received HMAC: "
)

start = time.perf_counter()

valid = verify_hmac(
    received_message,
    secret_key,
    received_hmac
)

end = time.perf_counter()

if valid:
    print("\nResult: MESSAGE IS AUTHENTIC")
```

```
        print("Integrity verification successful.")
    else:
        print("\nResult: MESSAGE IS NOT AUTHENTIC")
        print("Message may have been modified.")

    print(
        "Verification time: {:.6f} seconds".format(
            end - start
        )
    )


if __name__ == "__main__":
    main()
```

- **Sample Output — Valid Message**

```
HMAC-SHA256 MESSAGE AUTHENTICATION
------------------------------------------

Enter message: Transfer Rs.1000
Enter secret key: mysecretkey

Generated HMAC:
7c1e8f2b0d9a4c...

Generation time: 0.000021 seconds
```

MESSAGE VERIFICATION

Enter received message: Transfer Rs.1000

Enter received HMAC: 7c1e8f2b0d9a4c...

Result: MESSAGE IS AUTHENTIC

Integrity verification successful.

Verification time: 0.000018 seconds

- **Sample Output — Modified Message**

Suppose the original message was:

Transfer Rs.1000

An attacker changes it to:

Transfer Rs.9000

The HMAC no longer matches.

Result: MESSAGE IS NOT AUTHENTIC

Message may have been modified.

- **HMAC vs Simple Hashing**

Feature	**SHA-256 Hash**	**HMAC-SHA256**
Uses secret key	No	Yes
Detects accidental modification	Yes	Yes
Provides authentication	No	Yes
Resistant to message modification by attacker	Limited	Strong, assuming key remains secret

Feature	SHA-256 Hash	HMAC-SHA256
Output	Hash	Authentication code
Common use	File integrity/checksums	API authentication, secure protocols

---

Why Simple Hashing Is Not Enough

Suppose Alice sends:

Message = Pay ₹1000

Hash = SHA256(Message)

An attacker changes:

Pay ₹1000

to:

Pay ₹9000

The attacker can calculate:

SHA256(Pay ₹9000)

and replace the original hash.

The receiver may therefore have no way to know that the message was changed.

With HMAC:

Message + Secret Key

↓

HMAC

The attacker needs the secret key to create a valid HMAC for the modified message.

- **Security Analysis**

Integrity

HMAC detects whether the message has been modified.

Original Message

↓

HMAC A

Modified Message

↓

HMAC B

$A \neq B$

Therefore, the receiver can reject the modified message.

Authentication

HMAC provides **message-origin authentication** in the sense that a valid MAC indicates that the sender possessed the shared secret key.

However, because both parties share the same key, HMAC does **not provide non-repudiation**. Either party holding the key could have generated the HMAC.

Secret Key Requirement

The security of HMAC depends heavily on protecting the secret key.

If:

Secret Key → stolen

an attacker can generate valid HMACs.

Therefore, secret keys should never be hard-coded into publicly distributed applications.

- **Performance Evaluation**

HMAC-SHA256 is computationally efficient.

A simple experiment can measure:

start = time.perf_counter()

generated_hmac = generate_hmac(

```
    message,
    secret_key
)


end = time.perf_counter()
```

The time can then be calculated as:

Execution Time = End Time - Start Time

For ordinary messages, HMAC-SHA256 typically executes very quickly on modern computers.

The overhead compared with a plain SHA-256 hash is small, while the security benefits are significant when authentication is required.

- **Advantages**
- Provides message integrity.
- Provides authentication using a shared secret.
- Efficient and fast.
- Uses a well-established cryptographic construction.
- SHA-256 provides a strong underlying hash function.
- Suitable for APIs and communication protocols.

Limitations

- Both parties must securely share the secret key.
- Key compromise compromises authentication.
- Does not provide non-repudiation.
- Does not encrypt the message.
- HMAC only authenticates data; it does not provide confidentiality.

- **Conclusion**

The experiment successfully implements **HMAC-SHA256** for message authentication. Unlike a simple hash, HMAC uses a secret key, allowing the receiver to verify both the **integrity and authenticity** of a message. Any modification to the message results in a different HMAC and causes verification to fail.

- **Quick Comparison of the Two Experiments**

Criteria	**Q1: Password Hashing**	**Q2: HMAC**
Main concept	Password security	Message authentication
Algorithm	PBKDF2-HMAC-SHA256	HMAC-SHA256
Salt	Yes	No
Secret key	No	Yes
Key stretching	Yes	No
Brute-force protection	Yes	Not its purpose
Rainbow-table protection	Yes	Not its purpose
Integrity	Indirectly	Yes
Authentication	Password verification	Message authentication
Python libraries	hashlib, secrets, hmac	hmac, hashlib
Performance measurement	Hash/verification time	Generation/verification time

For your rubric

These two answers cover all five rubric areas well:

- **Understanding of Concepts (25%)** → Theory, security principles, comparisons
- **Experimental Design (30%)** → Complete Python implementation + testing
- **Use of Tools (20%)** → Python cryptographic libraries and timing measurement
- **Analysis of Results (15%)** → Attack resistance, performance and limitations

- **Documentation (10%)** → Aim, objectives, theory, algorithm, code, output, analysis and conclusion

For a submission, I recommend adding **screenshots of your actual Python execution output** under each program. This will strengthen the *Experimental Design* and *Documentation* portions of the rubric.